TEACHER VERSION. Do not hand out. Same booklet as the pupil copy, with the **answers in red**, the hints printed the right way up, and the 90 minute timeline below. Pupils get `Intro-to-Python-All-in-One.pdf` (no timings, no answers). Code: `code/ladder.py` and `code/calculator.py` (starters), `ladder_solution.py` and `calculator_solution.py`.

Intro to Python: 90 minute timeline

Slide numbers are for `slides/python-intro.pptx` (54 slides, speaker notes carry the same times).

Minutes	What happens	Slides	In the booklet
0–8	**Hook and intro.** Enigma machine, who cracked it (Turing). Today we learn the language computers are told things in.	1–3	Slides only
8–18	**Pupils: get started on Spyder.** Open Spyder, find the editor and console, run `print('hello world')`, then console maths. Circulate and fix start-up problems.	4–9	Parts 1 and 2
18–28	**Lecture.** Variables, strings, numbers, booleans, `type()`. Then functions: f(x), `def`, `return`, calling, two inputs.	10–22	Part 2 (variables, types), Part 3
28–50	**Function Ladder.** Solo or pairs on `ladder.py`, rungs 1 to 5 (stretch 6 and 7). Check test chips and trackers. Keen pupils carry on into Parts 5 and 6.	23–24	Part 4, `code/ladder.py`
50–57	**Together: model answers.** Build `add`, `subtract`, `multiply`, `divide` on the board. Gate: all four named exactly before the calculator.	25–34	Part 4 tracker
57–64	**Lecture.** `if`, `else`, comparisons, `elif`, `and`. Pupils run each and predict.	35–40	Part 5
64–84	**Build the calculator.** 4 min walk through the steps, then pupils build `calculator.py`. Test with `*`, 6, 7.	41–47	Part 6, `code/calculator.py`
84–90	**Show and recap.** Two or three pupils demo. What you built today, challenge: add `power`.	53–54	Tracker on page 1
Stretch	**Loops** (`for`, `while`) are not taught in this lesson. Slides 48–52 are for the next lesson. Keen pupils use Part 7.	48–52	Part 7, cheat sheet

If behind: cut the model-answer slides to `add` and `divide`, and show the calculator steps instead of building from scratch. **Watch for:** wrong function names (`substract`), missing `return`, `=` instead of `==`, missing colons and indents. Read the last line of the console.

Intro to Python

Your workbook for the whole lesson. Everything you need is in here. Keep it on your desk.

Name: _____ Class: _____

Today What is in this booklet

Seven Parts, from your first line of code to a working calculator, then a two page **Python cheat sheet** at the back. Work through the Parts in order.

Want to get ahead? You do not have to wait for the teacher. If you finish a Part, start the next one. Every Part is in this booklet, and there are stretch tasks at the end (Part 7 and the S tasks). The cheat sheet on the last two pages helps with all of it.

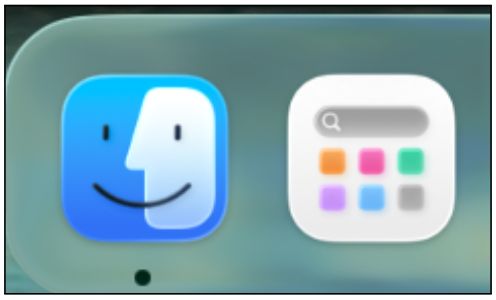
Tracker Tick each Part when you have finished it

- ☐ **Part 1** I opened Spyder and ran my first program
- ☐ **Part 2** I used the console as a calculator and made variables
- ☐ **Part 3** I can explain what a function is
- ☐ **Part 4** The Function Ladder: rungs 1, 2, 3, 4, 5 (circle the rungs you finished)
- ☐ **Part 5** I wrote an `if`, an `else` and an `elif`
- ☐ **Part 6** My calculator works
- ☐ **Part 7** Loops and stretch tasks (circle the ones you tried: S1 S2 S3 S4 S5)

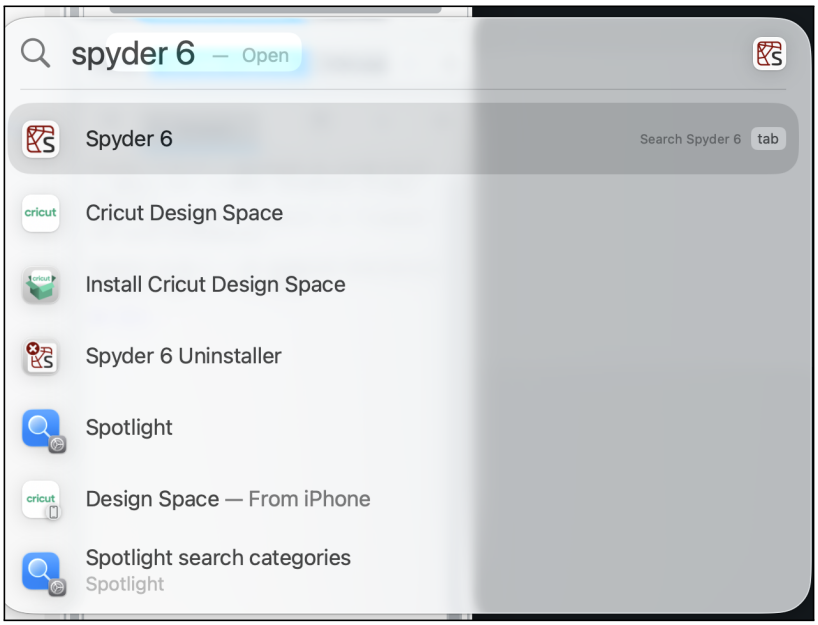
Three rules for coding. (1) Read the last line of the console if something breaks. It tells you which line went wrong. (2) Change one thing at a time, then run again. (3) Asking a neighbour is allowed. Copying their answer is not. Ask them *how*.

Part 1 Getting started in Spyder (Mac)

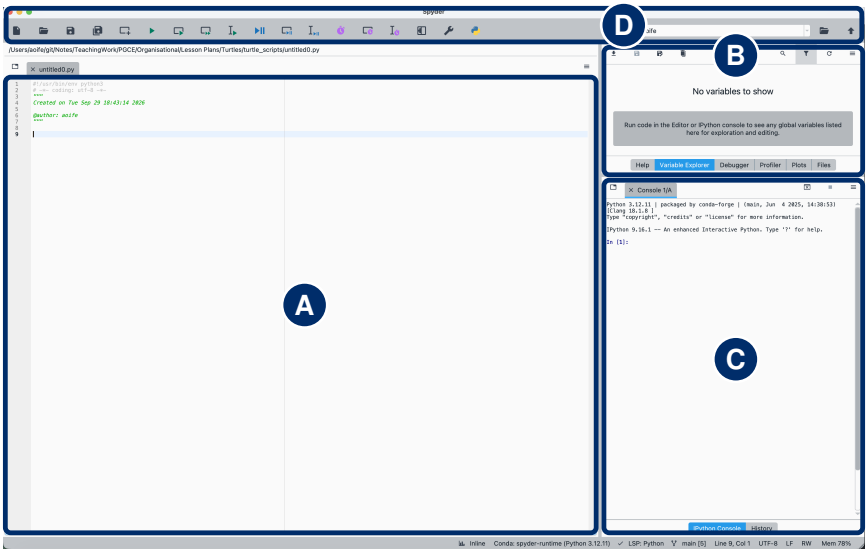
1 Open the search. Press **Cmd + Space**, or click the grid of squares in the dock.



2 Type "spyder" and press Enter. It can take 10–20 seconds to open. Be patient.



3 Find your way round. Spyder has three main areas and a row of buttons.



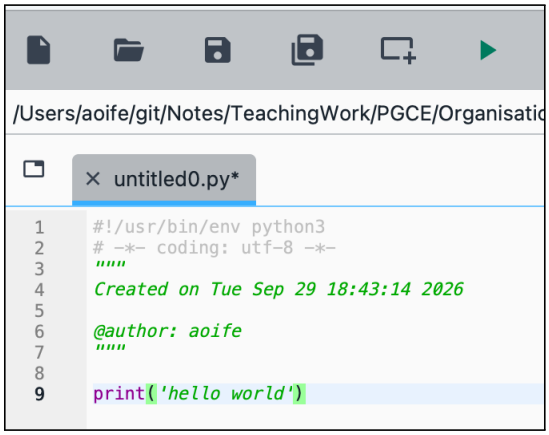
A **Editor** (left). You *write* your program here.

B **Variable Explorer** / **Plots** (top right). Shows what your program has stored, and graphs.

C **Console** (bottom right). Your program's *output* appears here. You can also type quick commands.

D **Toolbar** (top). The green triangle ▶ is **Run**.

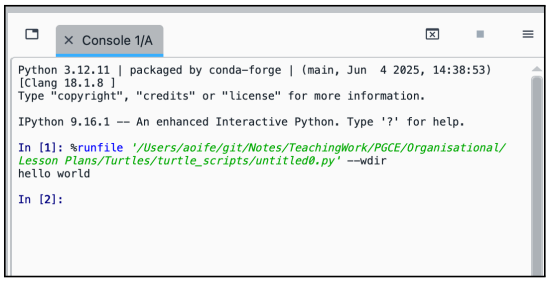
4 Write your first program. Click in the editor (A) and type this on a new line:



`print('hello world')`

The * next to the file name means "not saved yet".

5 Run it. Click the green ▶ button, or press **F5**. Look in the console (C).



You should see `hello world`. Your first program! Tick it off on page 1.

Part 2 Practise in the console

The console runs Python **instantly**, one line at a time. Click in the console (C), type the line, press Enter. **Predict first**, then check.

type this	I think it shows	it actually showed	✓
`2 + 2`	4	4	✓
`8 / 2`	4.0	4.0	✓
`7 * 6`	42	42	✓
`10 % 3`	1	1	✓
`2 + 3 * 4`	14	14	✓
`(2 + 3) * 4`	20	20	✓

`+` add `-` subtract `*` multiply `/` divide `%` the remainder after dividing

Order matters. What is different about the last two lines, and why?

`2 + 3 * 4` **gives 14 because Python multiplies before it adds. The brackets in** `(2 + 3) * 4` **make it add first, so it gives 20.**

Variables. A **variable** is a labelled box that holds a value. Type each line in the console. Write what it shows (or "nothing").

type this	it shows	what just happened?
`age = 16`	**nothing**	**stores 16 in the box called age**
`age`	**16**	**shows what is stored in age**
`score = 95`	**nothing**	**stores 95 in score**
`score = score + 5`	**nothing**	**adds 5 and stores 100 back in score**
`score`	**100**	**the new value replaced the old one**
`name = "Zendaya"`	**nothing**	**stores the text Zendaya**
`name`	**'Zendaya'**	**shows the text in name**

In Python, what does `=` mean? It does *not* mean "equals". Finish the sentence:

`=` means *store*: it puts the value on the right into the name on the left.

Types. Every value has a type. Type these and fill in the table.

type this	it shows	in plain words
`type(age)`	`<class 'int'>`	**a whole number**
`type(name)`	`<class 'str'>`	**text**
`type(1.75)`	`<class 'float'>`	**a number with a decimal point**
`type(True)`	`<class 'bool'>`	**True or False**

`int` is a whole number. `float` has a decimal point. `str` is text in quotes. `bool` is `True` or `False`. Notice the capital letters.

Your turn. Make three variables of your own, one of each of three different types. Write the lines you typed:

My three variables

Any three, for example: `age = 16` (int), `pet = "cat"` (str), `tall = True` (bool).

Part 3 Functions: a machine for maths

In maths, $f(x) = x + 1$ is a machine. Put a number in, get a number out. Python functions work the same way.

```
def add_one(x):
    return x + 1
```

1. `def` says "I am defining a function".
2. `add_one` is the name we chose.
3. `x` in the brackets is the **input** (a parameter).
4. `return` sends the answer back out.

Calling a function means using it. In the console, type `add_one(5)`. What does it show? 6 _____

Two inputs. Put them in the brackets with a comma between:

```
def add(a, b):
    return a + b
```

Type that into the *editor*, run the file (F5), then try `add(3, 4)` in the console. It shows 7 _____. Try `add(10, 20)` too.

Why bother? Write the maths once. Reuse it forever.

Part 4 The Function Ladder

Rules. Work solo or in a pair. Every function goes in one Spyder file called `ladder.py`. Your teacher gives you a starter copy with a comment on every rung. Run the file with **F5**, call the function in the console, and check it against its **test chip**. Tick the chip only when what you got matches. Each rung has **hints**, printed upside down on purpose: try Hint 1, have another go, and only read the next one if you still need it. Your teacher checks your tracker at the end.

1. **Rung 1: no input.** `lucky()` gives back the number 7. Write it in `ladder.py`.

`lucky()` expected: 7 I got: 7 _____ ☒

```
def lucky():
    return 7
```

Hints (printed upside down for pupils)

Hint 1: a function starts with the word `def`. **Hint 2:** after the colon the next line is indented. **Hint 3:** the way to send a value back is a word you met on the last page.

2 Rung 2: one input. `add_one(x)` gives back `x` plus 1. Then write **your own** one-input function (any name, any maths) and test it.

`add_one(5)` expected: 6 I got: **6** ☒

Name of your own function: **double** and a call that works: **double(4) gives 8**

```
def add_one(x):
    return x + 1
```

Hints (printed upside down for pupils)

Hint 1: look at the function in Part 3. Yours is almost the same. **Hint 2:** the line you need starts with `return`. **Hint 3:** after `return` comes a sum made from `x` and a number.

3 Rung 3: what is `return` FOR? Type this in the file, run it, then call `hello()` in the console.

```
def hello():
    greeting = "hi"      # no return here
```

code	what I think happens	what actually happened
`add_one(5)`	**6**	**6**
`hello()`	**many expect "hi"**	**nothing shows**

Why did `hello()` behave differently? Finish: "Without return, a function ..."

...does its work but gives nothing back. The console shows nothing because there is no `return` (it gives back `None`).

4 Rung 4: two inputs. `average(a, b)` gives back the average of `a` and `b`. `product(a, b)` gives back `a` times `b`.

`average(3, 5)` expected: 4.0 I got: **4.0** ☒

`product(6, 7)` expected: 42 I got: **42** ☒

```
def average(a, b):
    return (a + b) / 2
def product(a, b):
    return a * b
```

Hints (printed upside down for pupils)

Hint 1: you need `(a, b)` in the brackets of the `def`. **Hint 2:** the average is the total divided by how many numbers there are. **Hint 3:** brackets in Python work like brackets in maths. Add first, then divide.

5 Rung 5: the four functions the calculator needs. The names must be **exactly** `add`, `subtract`, `multiply`, `divide`. Each takes `(a, b)`. One wrong letter and the calculator cannot find it.

```
def add(a, b):       return a + b
def subtract(a, b):  return a - b
def multiply(a, b):  return a * b
def divide(a, b):    return a / b
```

Run the file once, then check all four in the console:

`add(2, 3)` expected: `5` — I got: **5** ☒

`subtract(10, 4)` expected: `6` — I got: **6** ☒

`multiply(6, 7)` expected: `42` — I got: **42** ☒

`divide(9, 2)` expected: `4.5` — I got: **4.5** ☒

Hints (printed upside down for pupils)

Hint 1: every function has the same shape. Only the maths symbol changes. **Hint 2:** Python's symbols are `+ - * /`. Match each to its name. **Hint 3:** you can use `a` and `b` in the `return` line, just like in maths.

Tracker. Tick only when the console value matches the test chip.

function	test worked?	notes
`add`	✓	**all test chips match**
`subtract`	✓	**all test chips match**
`multiply`	✓	**all test chips match**
`divide`	✓	**all test chips match**

Gate. All four must be in one file and run before the calculator (Part 6) will work. The calculator uses them by these exact names.

6 Rung 6: calls inside calls. Guess first, then run each line in the console.

Inside-out rule: do the bracket nearest the middle first, then use its answer in the outer call.

`add(multiply(3, 4), 2)` — I think: **14** — I got: **14**

`divide(add(6, 8), 2)` — I think: **7.0** — I got: **7.0**

7 Rung 7 (stretch). `power(a, b)` gives `a` to the power `b`. Put it in `ladder.py`.

`power(2, 10)` expected: `1024` — I got: **1024** ☒

```
def power(a, b):
    return a ** b
```

Hints (printed upside down for pupils)

Hint 1: same shape as the other four functions. **Hint 2:** the Python symbol for "to the power of" is two stars in a row. **Hint 3:** `a ** b` means multiply `a` by itself `b` times.

Part 5 Making decisions with `if`

Code needs to make decisions too. *If it is raining, bring an umbrella. Otherwise, don't.* Type this into the editor and run it. Then change `temperature` and run it again.

```
temperature = 30
if temperature > 25:
    print("It's hot!")
else:
    print("It's chilly")
```

Watch out: the colon `:` after the condition, and the indent on the line underneath. Python needs both.

Predict, then check. When `temperature` is:

`temperature = 30` I think: **It's hot!** I got: **It's hot!**

`temperature = 10` I think: **It's chilly** I got: **It's chilly**

`temperature = 25` I think: **It's chilly** I got: **It's chilly**

Comparing. Fill in what each one means.

`==`	**equal to**	`>=`	**greater than or equal to**
`!=`	**not equal to**	`<=`	**less than or equal to**
`>`	**greater than**	`<`	**less than**

A double `==` *asks* "are these equal?". A single `=` *stores* a value. Mixing them up is the most common mistake.

More than two choices: `elif`. Python checks each condition from the top and stops at the first true one. Predict what each score prints, then run it.

```
score = 72
if score >= 90:
    print("A")
elif score >= 70:
    print("B")
elif score >= 50:
    print("C")
else:
    print("Needs work")
```

`score = 72` I think: **B** I got: **B**

`score = 95` I think: **A** I got: **A**

`score = 40` I think: **Needs work** I got: **Needs work**

Combining: `and` means both sides must be true. Write a program that prints `You're a teenager!` when `age` is from 13 to 19.

Your code

```
age = 16
if age >= 13 and age <= 19:
    print("You're a teenager!")
```

Hints (printed upside down for pupils)

Hint 1: you need one `if` and one `print`. **Hint 2:** the condition has two comparisons joined by `and`. **Hint 3:** one comparison uses >= and the other uses <=.

Part 6 Build the calculator

Variables + functions + `if` statements, together. You will write `calculator.py`. It asks which sum you want, asks for two numbers, then shows the answer. Your teacher gives you a starter file with the steps in order.

1 Step 1: the four functions. Copy your four functions from `ladder.py` to the top of `calculator.py`. The names must match exactly.

2 Step 2: ask the user. `input()` always gives back *text*. `float()` turns text into a number. Fill in the three lines.

```
operation = input("Pick: + - * / ")
num1 = float(input("First number: "))
num2 = float(input("Second number: "))
```

Hints (printed upside down for pupils)

Hint 1: the words in the quotes are what the user sees as the question. **Hint 2:** the first should offer the four symbols, like `Pick: + - * /`. **Hint 3:** the other two just ask for the first number and the second number.

3 Step 3: decide what to do. One branch for each symbol, each calling the right function. Fill in the blanks.

```
if operation == "+":
    result = add(num1, num2)
elif operation == "-":
    result = subtract(num1, num2)
elif operation == "*":
    result = multiply(num1, num2)
elif operation == "/":
    result = divide(num1, num2)
```

Hints (printed upside down for pupils)

Hint 1: the first branch is already done. Copy its pattern. **Hint 2:** each symbol goes with the function of the same meaning. **Hint 3:** the symbol inside the quotes is a piece of text, so the divide symbol needs quote marks too.

4 Step 4: show the result. An **f-string** drops a variable into text using curly brackets.

```
print(f"Answer: {result}")
```

Run it! Press F5 and answer the three questions in the console. Test it and write down what you got:

you type	I think	it showed	✓
`*` then `6` then `7`	**Answer: 42.0**	**Answer: 42.0**	✓
`/` then `20` then `4`	**Answer: 5.0**	**Answer: 5.0**	✓
`-` then `9` then `12`	**Answer: -3.0**	**Answer: -3.0**	✓

Nothing happens, or an error? Read the last line of the console. Check the function names, the colons, the indents, and that your four functions are really at the top of the file.

Part 7 Loops, and stretch tasks

Loops: do it again and again. *Don't repeat yourself.* Predict first, then run.

```
for i in range(5):
    print(i)
```

I think it prints: 0 1 2 3 4 It printed: 0 1 2 3 4

```
snacks = ["pizza", "chips", "soda"]
for snack in snacks:
    print(snack)
```

I think it prints: pizza chips soda It printed: pizza chips soda

```
count = 0
while count < 3:
    print(count)
    count += 1
```

I think it prints: 0 1 2 It printed: 0 1 2

`for` is for a known number of times. `while` keeps going until something changes. What happens if you delete `count += 1`? (Press the red stop square if it will not stop.) runs forever, printing 0

Stretch tasks. Pick any. Do them in a new file or in `ladder.py`. Tick them off.

- ☐ **S1** Add `power(a, b)` to the calculator, with a new `elif` for the symbol `**`.
- ☐ **S2** Write `average(a, b)` into the calculator as a fifth operation.
- ☐ **S3** Write a function `ticket(age)`. Under 13 it gives back 5, otherwise 10. Test it with three ages.
- ☐ **S4** Use a `for` loop to print the 7 times table, from 1×7 to 10×7.
- ☐ **S5** Use a loop and `+=` to add up the numbers from 1 to 100. Make a prediction first.

```
# S1  def power(a, b): return a ** b   and a new branch:  elif operation == "**": result = power
    (num1, num2)
# S2  def average(a, b): return (a + b) / 2   and a new elif for the chosen symbol
def ticket(age):                        # S3
    if age < 13:
        return 5
    else:
        return 10
for i in range(1, 11):                  # S4
    print(i * 7)
total = 0                               # S5  prints 5050
for n in range(1, 101):
    total += n
print(total)
```

Hints (printed upside down for pupils)

S1: the calculator already has the pattern. You need a new function and a new branch. **S3:** one `if` and one `else`, each with its own `return`. **S4:** `range(1, 11)` counts from 1 to 10. **S5:** start a variable at 0, then add the loop counter to it each time round.

Finished everything? Teach a neighbour one thing you learned. Then invent your own function and give it to them to test.

Python cheat sheet

Everything from the lesson, to keep next to you

1 Maths in the console

`+` add `*` multiply `%` remainder
`-` subtract `/` divide `()` brackets first

```
>>> 2 + 3 * 4        # multiply first
14
>>> (2 + 3) * 4      # brackets first
20
>>> 10 % 3           # 1 slice left over
1
>>> 5 / 3
1.6666666666666667
```

2 Variables

A variable is a **labelled box** that holds a value.

```
age = 16                # int (whole)
height = 1.75           # float (decimal)
name = "Zendaya"        # str (text)
is_hungry = True        # bool

score = score + 5       # variables can
    change
type(age)               # check the type
```

= means **store**, not "equals".

3 Functions

Like $f(x) = x + 1$ in maths: something goes in, something comes out.

```
def add(a, b):          # def = define
    return a + b        # return = send back

add(3, 4)               # calling it gives 7
```

`def` starts a function
`add` name you choose
`(a, b)` the inputs (parameters)
`return` the output

```
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    return a / b        # always a float
```

4 Making decisions: if

```
score = 72
if score >= 90:
    print("A")
elif score >= 70:
    print("B")
elif score >= 50:
    print("C")
else:
    print("Needs work")
```

Python checks top to bottom and stops at the **first** true one.

`==` equal to `>` greater than
`!=` not equal to `<` less than
`>=` greater or equal `<=` less or equal

```
if age >= 13 and age <= 19:    # both true
    print("You're a teenager!")
```

Use == to *compare*. A single = stores.

5 Loops

for: a known number of times.

```
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

snacks = ["pizza", "chips", "soda"]
for snack in snacks:
    print(snack)
```

while: until something changes.

```
count = 0
while count < 3:
    print(count)
    count += 1          # forget it =
    forever!
```

6 Input and output

```
num = float(input("Number: "))
print(f"Answer: {num * 2}")
```

`input()` always gives back *text*, so `float()` turns it into a number. An `f"..."` string drops variables into text with curly brackets.

7 Putting it together: calculator.py

```
operation = input("Pick: + - * / ")
num1 = float(input("First number: "))
num2 = float(input("Second number: "))

if operation == "+":
    result = add(num1, num2)
elif operation == "-":
    result = subtract(num1, num2)
elif operation == "*":
    result = multiply(num1, num2)
elif operation == "/":
    result = divide(num1, num2)

print(f"Answer: {result}")
```

How it works

1. Write the 4 functions.
2. Ask the user for a sum.
3. `if`/`elif` picks the right function.
4. Print the answer.

Running it

```
Pick: + - * / *
First number: 6
Second number: 7
Answer: 42.0
```

8 The four types of value

Type	Meaning	Examples	Made with
`int`	whole number	`16`, `-3`, `100`	`age = 16`
`float`	has a decimal point	`1.75`, `4.0`	`height = 1.75`
`str`	text (in quotes)	`"Zendaya"`	`name = "Zendaya"`
`bool`	only two values	`True`, `False`	`is_hungry = True`

Common mistakes

- `=` vs `==`: store vs compare.
- Forgetting the `:` after `if`, `else`, `def`, `for`, `while`.
- Forgetting to **indent** the lines inside.
- Text needs **quotes**: `"hello"`, not `hello`.
- A `while` loop that never changes its condition runs forever (stop it with the red square in the console).

Stuck?

Read the **last line** of the console. It tells you which line went wrong. Change one thing at a time, then run again.